Storage Class

Storage Class

A storage class is used to describe about the features of a variable or function. These features include place of storage, default initial value, scope and life-time of a variable.

ScannedbyCamScanner

A variable's storage class gives the following information:-

1) where the variable would be stored
2) what will be the initial value of the variable, if initial value is not assigned (i.e. default initial value).
3) what is the scope of the variable (scope of a variable means in which functions the value of the variable would be available, i.e. the variable will be accessible by name).
4) what is the life of the variable (life of a variable means how long would the variable exist).

There are 4 storage classes in C:-

1) Automatic Storage class
2) Register Storage class
3) Static Storage class
4) External storage class

1) Automatic Storage class:-

i) Storage — Memory
ii) Default Initial value — Garbage value
iii) Scope — Local to the block in which the variable is defined.
iv) Life — Till the control remains within the block in which the variable is defined.

Example:-

```
void main()
{
    auto int i=1;
    {
        auto int i=2;
        {
            auto int i=3;
            printf("%d", i);
        }
        printf("%d", i);
    }
    printf("%d", i);
}
```

Output:- 3 2 1

ScannedbyCamScanner

In this program, the three i's are defined in different blocks. All 3 i's are available to the innermost printf(). This innermost printf() prints 3 because when all three i's are available, the one which is most local i.e. nearest to printf() is given priority. Once the control comes out of the ~~block~~ innermost block, the variable i with value 3 is lost, and hence the i in the second printf() refers to i with value 2. Similarly, when the control comes out of the next innermost block, the third printf() refers to i with value 1.

2) Register Storage Class

i) Storage — CPU Registers.

ii) Default Initial Value — Garbage value.

iii) Scope — Local to the block in which the variable is defined.

iv) Life — Till the control remains within the block in which the variable is defined.

If a variable is used at many places in a program, it is better to declare its storage class as register. This is because a value stored in a CPU register can always be accessed faster than the one stored in memory.

Example:-

```
void main()
{
    register int i;
    for(i=1; i<=10; i++)
        printf("%d", i);
}
```

A good example of frequently used variables is loop counters.

ScannedbyCamScanner

3) Static Storage Class :-

i) Storage — Memory

ii) Default Initial value — Zero

iii) Scope — Local to the block in which the variable is defined.

iv) Life — Value of the variable persists between different function calls.

The difference between automatic variables and static variables is that static variables do not disappear when the function is no longer active. Their values persist. If the control comes back to the same function again the static variables have the same values they had last time around.

Example of a program to show the difference between auto & static variable

```
void main()
{
    increment();
    increment();
    increment();
}
void increment()
{
    auto int i=1;
    printf("%d\n", i);
    i=i+1;
}
```

Output

1
1
1

```
void main()
{
    increment();
    increment();
    increment();
}
void increment()
{
    static int i=1;
    printf("%d\n", i);
    i=i+1;
}
```

Output

1
2
3

When variable i is auto, each time the function increment() is called, i is re-initialized to 1. When the

ScannedbyCamScanner

function terminates, i vanishes and its new value of 2 is lost. But, if i is static, it is initialized to 1 only once. It is never initialized again. During the first call to increment(), i is incremented to 2. The next time increment() is called, i is not re-initialized to 1. Its old value 2 is still available. So, now 2 gets printed and then i=i+1 adds 1 to i to get a value of 3 which gets printed the next time.

4) External Storage Class

Storage — Memory

Default Initial Value — Zero

Scope — Global

Life — As long as the program's execution does not come to an end.

External variables are declared outside all functions, yet they are available to all functions that use them.

Example:-

```
int x=10;
void main()
{
    extern int y;
    printf("\n %d %d", x,y);
}
int y=20;
```

Here, both x & y are global variables. Since both of them have been defined outside all the functions both enjoy external storage class.

ScannedbyCamScanner

V. IMP

Storage class	Storage	Default Initial Value	Scope	Life
1) auto	Memory	Garbage value.	Local to the block in which the variable is defined.	Till the control remains within the block in which the variable is defined.
2) register	CPU Registers.	Garbage value.	"	"
3) static	Memory	Zero (0)	"	Value of the variable persists between different function calls.
4) extern	Memory	Zero (0)	Global	As ~~soon~~ long as the program's execution does not come to an end.

ScannedbyCamScanner